Week 1 Reading: Process Prediction

1. Definition of the prediction map

A surrogate predicts outputs from specified inputs over a declared domain. A label is a reference output paired with one input row.

Role	This prediction task
Inputs	T: 300–360 K; τ: 1–10 min; C_Af: 0.8–1.5 mol/L
Outputs	C_A in mol/L; conversion X, dimensionless
Fixed context	Ideal, constant-volume, isothermal CSTR; first-order A → B; fixed rate parameters
Reference	Steady-state material-balance solution

Vary feed concentration within the declared domain while keeping reaction assumptions and units the same across rows.

A temperature name and numerical value need units and an operating range to define the prediction task.

2. Derive the steady-state map

The ideal CSTR is perfectly mixed, isothermal, and constant-volume, with first-order reaction A → B. The balance is accumulation = inlet − outlet − reaction:

$$\frac{dC_A}{dt} = \frac{C_{Af} - C_A}{\tau} - k\left(T\right)C_A$$

At steady state, set the derivative to zero. Multiply by residence time, move the concentration terms to the same side, and factor out outlet concentration. Dividing by the positive factor gives:

$$C_A = \frac{C_{Af}}{1 + k\left(T\right)\tau}, \quad X = 1 - \frac{C_A}{C_{Af}} = \frac{k\left(T\right)\tau}{1 + k\left(T\right)\tau}$$

At 330 K, the rate constant is 0.2/min. At 5 min residence time, the product of rate and residence time is 1, so conversion is 0.5. A 1.2 mol/L feed gives a 0.6 mol/L outlet.

Conversion is independent of feed concentration in this first-order model.

A steady-state calculation, step by step

At 330 K, $\tau=5$ min, and $C_{Af}=1.2$ mol/L, $k=0.2$/min. Treat concentration as the unknown and rearrange the balance.

Step	Calculation	Interpretation
Zero accumulation	$0=(1.2-C_A)/5-0.2C_A$	Flow and reaction balance
Multiply by residence time	$0=1.2-C_A-1.0C_A$	Every term now in mol/L
Collect concentration	$2C_A=1.2$	Add coefficients of the unknown
Concentration and conversion	$C_A=0.6$; $X=1-0.6/1.2=0.5$	Half of A converted

Substitution gives a net flow term $(1.2-0.6)/5=0.12$ mol/(L·min) and reaction consumption $0.2\times0.6=0.12$. Their equality explains why concentration no longer changes.

3. Build independent train, validation, and test data

Assume the following rate law, with T in K and k in 1/min:

$$k\left(T\right) = 0.2\exp\left[6000\left(\frac{1}{330} - \frac{1}{T}\right)\right]$$

Sample inputs independently and uniformly inside the declared box; compute each label with the steady-state equations. Seed 42 fixes a reproducible sample.

Split	Rows	Role
Train	400	Fit coefficients
Validation	120	Compare model choices
Test	160	Final held-out evaluation

The provided basis is fixed in advance. If comparing another basis or capacity, use validation and leave test untouched.

4. Scale inputs and fit ten features

$$\tilde{x}_j = 2\frac{x_j - l_j}{u_j - l_j} - 1$$

At [330 K, 5 min, 1.2 mol/L], the scaled input is approximately [0, −0.111111, 0.142857]. The scaling uses declared bounds, not statistics computed from test data.

The feature row contains 1 constant + 3 inputs + 3 squares + 3 pairwise products = 10 features. For 400 train rows, Φ has shape (400, 10), Y (400, 2), and B (10, 2).

$$\min_B \|\Phi B - Y\|_F^2, \quad \hat{y} = \varphi\left(x\right)B$$

`np.linalg.lstsq` fits B. Keep B fixed when predicting new input rows.

NumPy least squares fits a coefficient vector for each output column of Y. The notation ‖ΦB−Y‖²_F means summing squared errors over all rows and columns, rather than requiring an exact fit to all 400 rows.

Fitting least-squares coefficients to three points

Use dimensionless data x=[−1,0,1], y=[1,2,2] and model b+ax. No line passes through all three points, so choose coefficients that minimize the sum of squared errors.

For errors $e_i=b+ax_i-y_i$, differentiation gives $2\sum e_i=0$ for b and $2\sum x_i e_i=0$ for a. Substituting $\sum x_i=0$, $\sum x_i^2=2$, $\sum y_i=5$ and $\sum x_i y_i=1$ gives 3b=5 and 2a=1.

x	Target y	Prediction 5/3+x/2	Error
−1	1	7/6	1/6
0	2	5/3	−1/3
1	2	13/6	1/6

```
import numpy as np
x = np.array([-1., 0., 1.])
phi = np.column_stack((np.ones(3), x))
coef = np.linalg.lstsq(phi, [1., 2., 2.], rcond=None)[0]
prediction = phi @ coef  # [7/6, 5/3, 13/6]
```

coef=[b,a] and the squared-error sum is 1/6. The CSTR fit extends the feature matrix to ten columns and applies the same calculation to each output column. `lstsq` solves the least-squares problem without explicitly forming an inverse.

From one input row to ten features

Scaling [330, 5, 1.2] with the declared bounds gives [a,b,c]=[0,−1/9,1/7]. Inputs with different units and magnitudes move to the same −1 to 1 interval.

Feature group	Values for this row
Constant and inputs	1, 0, −0.111111, 0.142857
Squares a^2,b^2,c^2	0, 0.012346, 0.020408
Products ab,ac,bc	0, 0, −0.015873

```
phi = features(np.array([[330., 5., 1.2]]))
phi_train = features(train_x)
coef = np.linalg.lstsq(phi_train, train_y, rcond=None)[0]
prediction = phi @ coef
```

Shapes are (1,10), (10,2), and (1,2) for phi, coef, and prediction. Matrix multiplication weights and sums feature columns separately for each output. Preserve the training feature order at new inputs.

The residence-time column is 2×(5−1)/(10−1)−1=−1/9; the feed column is 2×(1.2−0.8)/(1.5−0.8)−1=1/7. Squared features square these transformed values, not the physical values 5 and 1.2.

5. Prediction accuracy and output consistency

$$\mathrm{RMSE}_j = \sqrt{\frac{1}{N}\sum_{n=1}^{N}\left(\hat{y}_{nj} - y_{nj}\right)^2}$$

For each output, subtract the reference from the prediction, square, average over the test rows, and take the square root. Concentration RMSE is in mol/L; conversion RMSE is dimensionless.

The seed-42 baseline gives concentration RMSE 0.032623 mol/L and conversion RMSE 0.025608. Inspect individual errors as well as the averages.

$$r = \hat{C_A} + C_{\mathrm{Af}}\hat{X} - C_{\mathrm{Af}}$$

For feed 1.2, prediction C_A = 0.65 and X = 0.45 gives r = 0.65 + 1.2 × 0.45 − 1.2 = −0.01 mol/L. The predicted pair is not exactly consistent with the defined conversion relation.

From residuals to RMSE and consistency

Compare reference concentrations [0.60,0.40] mol/L with predictions [0.65,0.38] mol/L.

Calculation	Result
Prediction minus reference	[0.05,−0.02] mol/L
Square	[0.0025,0.0004] (mol/L)²
Average	0.00145 (mol/L)²
Square root	RMSE≈0.038079 mol/L

```
error = prediction - truth
rmse = np.sqrt(np.mean(error**2, axis=0))
```

axis=0 averages input rows and retains output columns. Averaging both outputs together would mix concentration with dimensionless conversion. Separately, C_A=0.65, X=0.45, C_Af=1.2 gives 0.65+1.2×0.45−1.2=−0.01 mol/L. Accuracy and the relation between outputs need separate checks.

6. Answer checks and readings

Try first: at 330 K, 10 min, and feed 1.2 mol/L, what are C_A and X? How many regression coefficients are fitted for two outputs? Why can a low training error be misleading?

Answer check: $k\tau = 2$, so C_A = 1.2/3 = 0.4 mol/L and X = 2/3. B contains 20 coefficients: ten features for each of two outputs. Training evaluates the same points used for fitting; independent errors can be larger.

Check your calculations in the Week 1 lab. Optional background: affine maps and least-squares problems in Chapters 2 and 4 of Boyd and Vandenberghe, *Convex Optimization* (official book PDF).